Sistemas Operacionais – Centro FAG

Sincronização de Processos

Análise do problema da Montanha-Russa

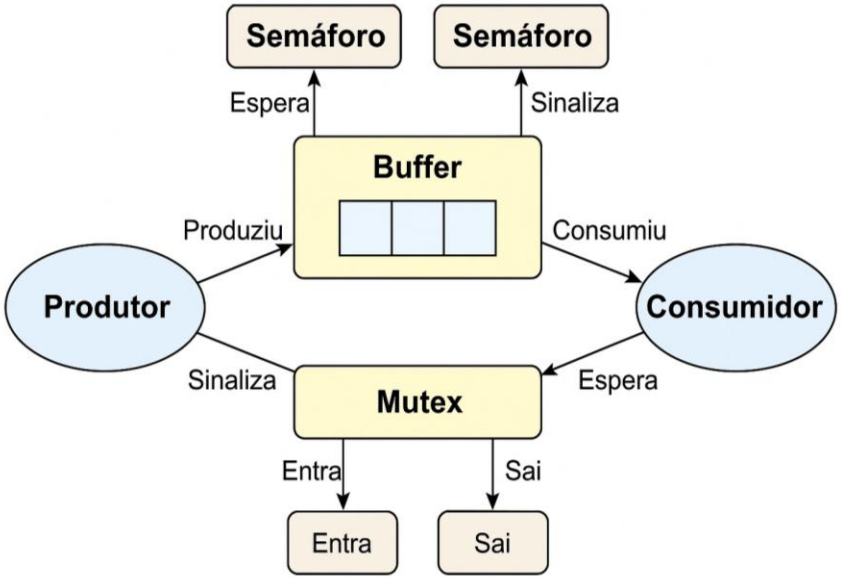
Andrei Gustavo de Lima Macedo, Caroline Balko Piovesam e Pedro Henrique Hideo Makiyama

O que é sincronização entre processos? Em que contexto é relevante para Sistemas Operacionais?

- **Definição:** Coordenação de múltiplas threads ou processos concorrentes.
- **Objetivo:** Garantir acesso seguro e ordenado a recursos ou memórias compartilhadas.
- **Contexto no SO:** O escalonador alterna a CPU rapidamente (*interleaving*).
- **Relevância:** Sem controle, essa troca rápida gera sobreposição de operações e corrompe o estado do sistema.

O que é um mutex e o que é um semáforo?

- **Mutex (Mutual Exclusion):**
  - Funciona como uma "chave única".
  - Garante exclusão mútua: apenas a thread dona da chave entra na seção crítica.
- **Semáforo:**
  - Funciona como um "contador de vagas" (0, 1, 2...).
  - Controla o acesso a recursos limitados ou sinaliza a conclusão de tarefas entre threads diferentes.



Qual a relação do problema escolhido por vocês com a dinâmica real de um SO?

- **Processamento em Lotes (*Batch Processing*):** O carrinho agrupa tarefas (passageiros) para processar de uma só vez.
- **Gerenciamento de Buffers de I/O:** O sistema acumula requisições até um limite de eficiência.
- **Otimização de Recursos:** O recurso pesado (carrinho) só é ativado quando o lote (lotação máxima) está completo, bloqueando alterações durante o processamento.

O que pode acontecer se você não estiver atento a sincronização de processos?

- **Condições de Corrida (*Race Conditions*):** Resultados imprevisíveis dependendo da ordem de execução.

- **Corrupção de Dados:** Contadores errados e atualizações perdidas (ex: carrinho partindo com capacidade excedida).

- **Deadlock:** Travamento total do sistema onde todas as threads ficam esperando umas pelas outras.

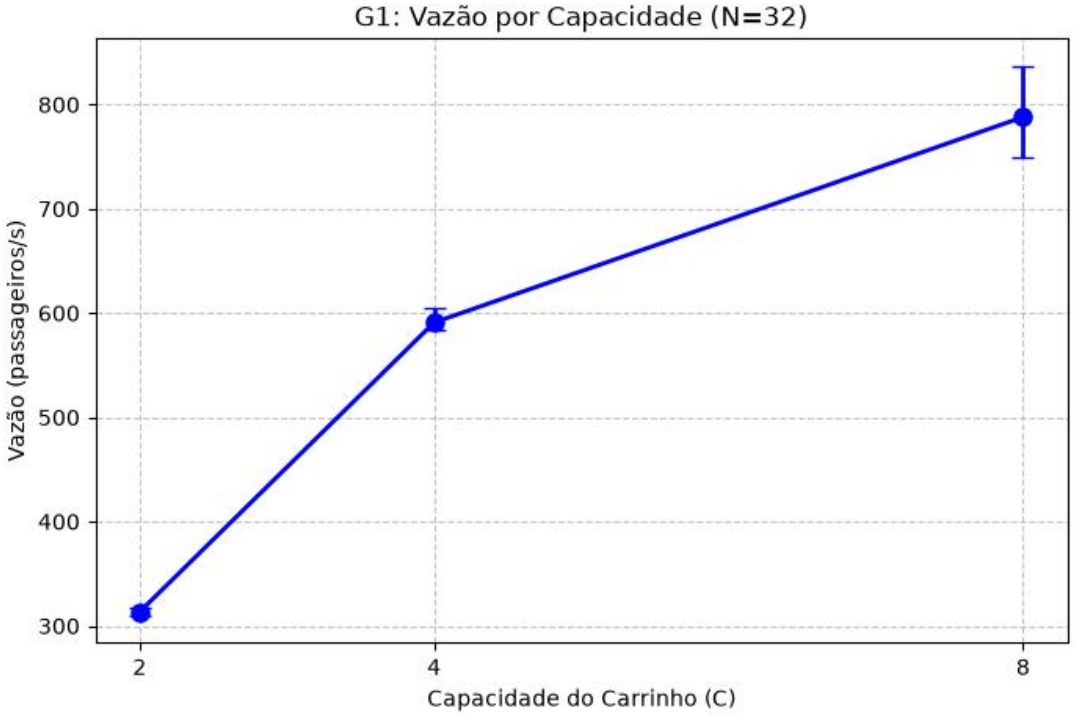
- **Starvation:** Uma thread (passageiro) sendo adiada indefinidamente, sem nunca conseguir executar.

Existem alternativas à solução que escolheram? A solução escolhida segue os bons princípios para problemas de sincronização? Quais são esses princípios?

- **Alternativa:** O código atual usa Variáveis de Condição e Mutex, mas poderia ser feito utilizando Semáforos Contadores.

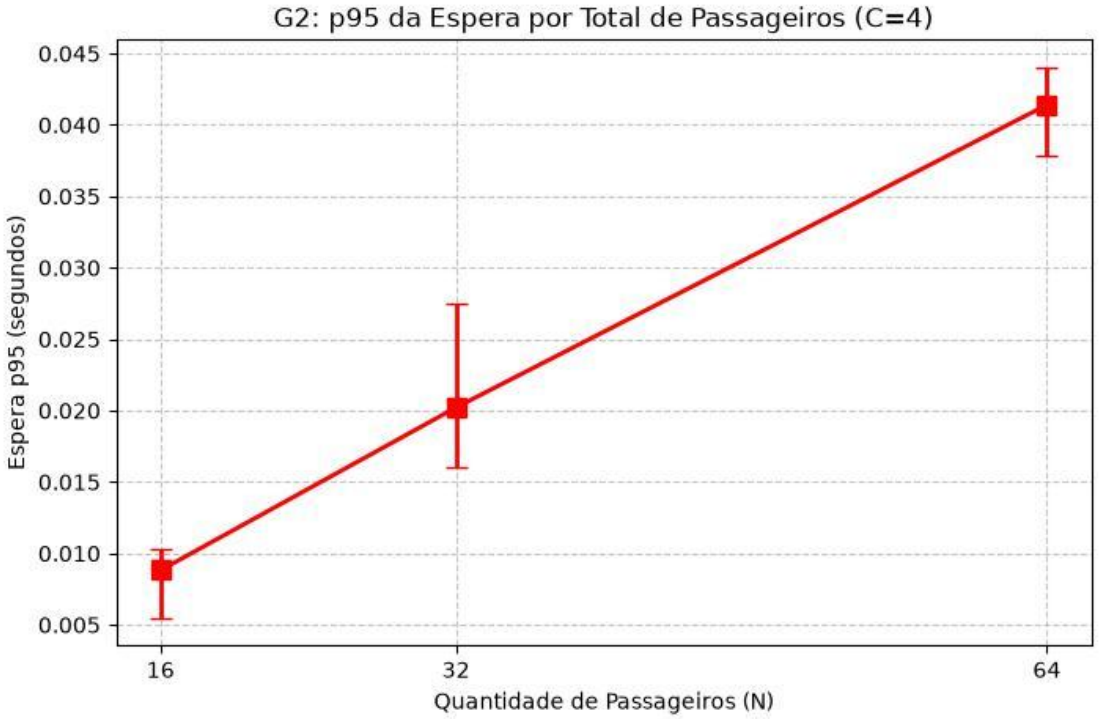
- **Os 3 Pilares Respeitados na Solução:**
  - Exclusão Mútua: Uso do Mutex para proteger as variáveis globais de lotação.
  - Progresso: Uso de "espera bloqueante" (cond_wait) em vez de gastar CPU em laços vazios (busy waiting).
  - Espera Limitada: Os gráficos provam que o p95 da espera é estável e finito, garantindo que ninguém sofra starvation.

Gráfico 1 (G1 - Vazão por Capacidade)



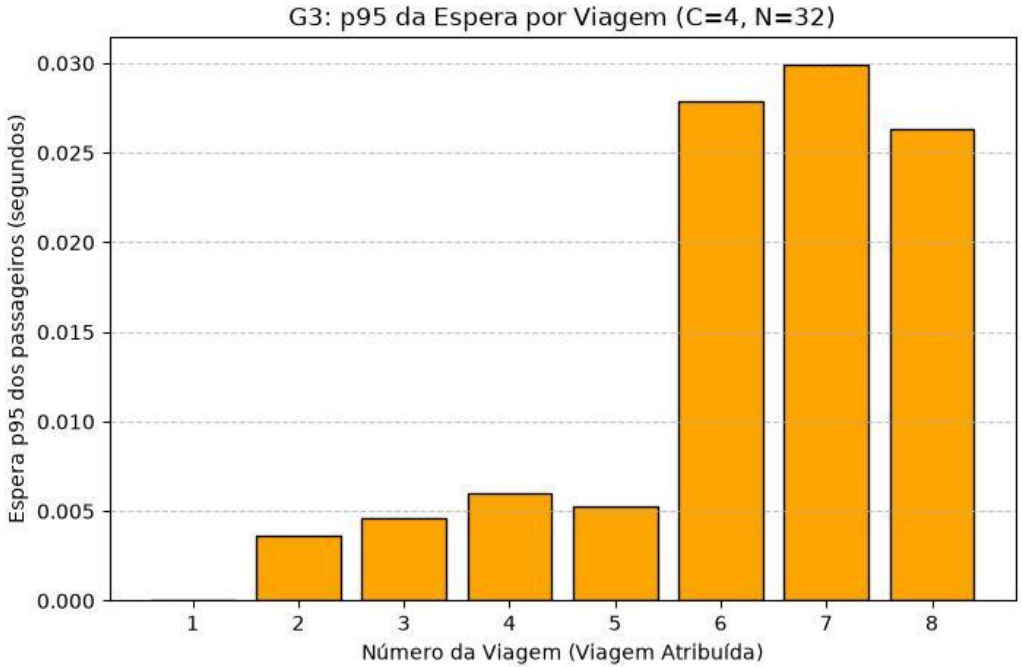
Como o tempo de passeio custa sempre 5 milissegundos, colocar carrinhos maiores dilui esse tempo e aumenta a produtividade do parque.

Gráfico 2 (G2 - p95 da Espera por Total de Passageiros)



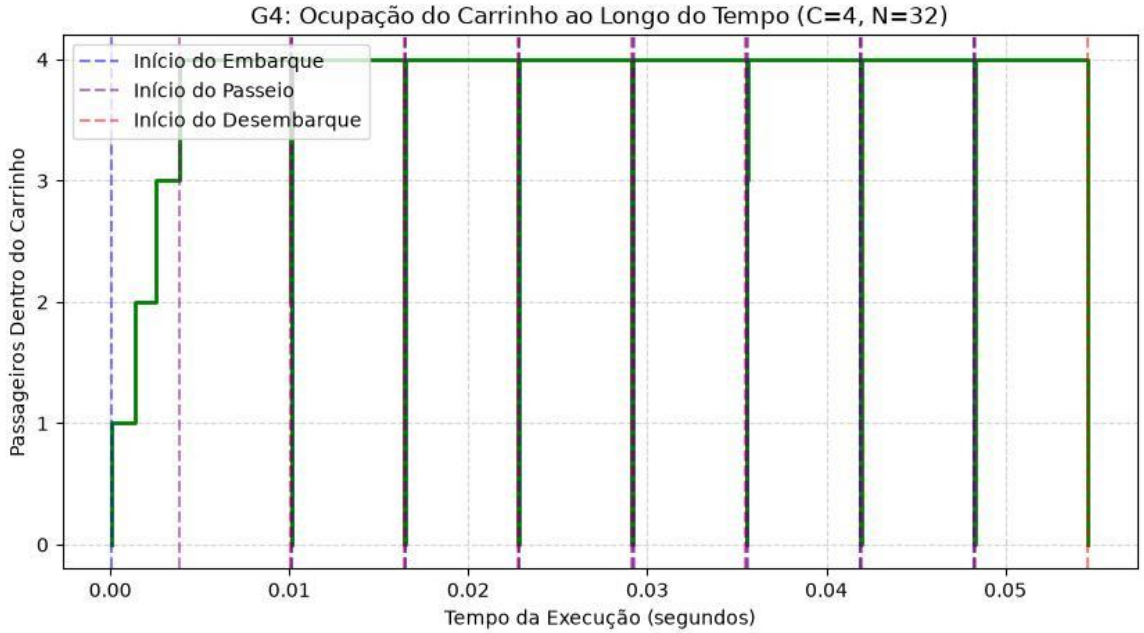
Torna o gargalo evidente. Um volume maior de pessoas acumuladas na fila piora o engarrafamento e aumenta a demora para conseguir embarcar.

Gráfico 3 (G3 - p95 da Espera por Viagem)



Mostra o "efeito bola de neve". As últimas viagens demoram mais porque as pessoas chegam na fila mais rápido (1ms) do que o carrinho consegue atender (5ms).

Gráfico 4 (G4 - Ocupação do Carrinho ao Longo do Tempo)



Este gráfico de degraus valida visualmente o cumprimento rigoroso de todas as regras do sistema. Demonstra que o carrinho só arranca com a lotação máxima e mantém-se fechado durante a viagem. Por fim, a linha cai para zero, comprovando o desembarque total antes do embarque seguinte.